

Plan d'optimisation du cycle de release — équipe Orion

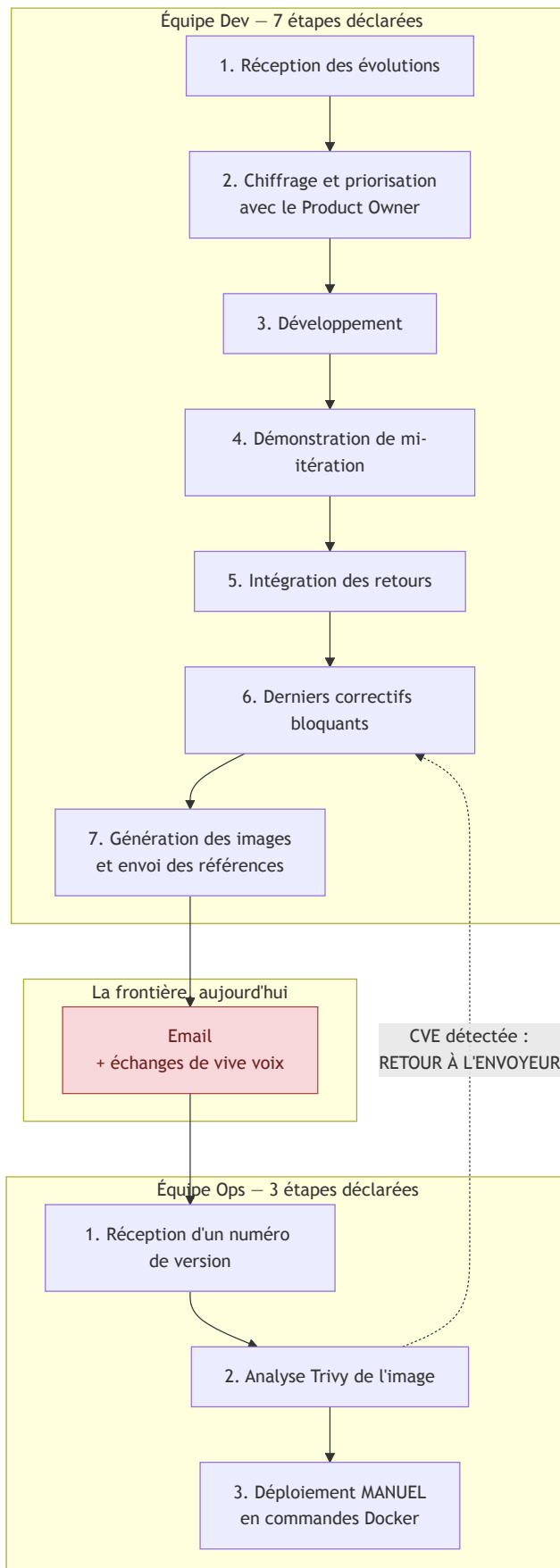
Livrable 2/2 de l'itération. Ce document part du cycle décrit par les deux équipes dans leurs sondages, nomme ce qui coûte, et propose un ordre de travaux avec un porteur, un effort et une preuve d'atteinte pour chaque action.

Il se lit avec son jumeau, [schema-architecture.md](#), qui décrit la cible. Ici on décrit **le chemin**.

Principe directeur. Une optimisation qui ne se mesure pas est une opinion. Chaque action porte donc une preuve observable — un job qui échoue, un chiffre qui bouge, une commande qui rend un résultat — et non un état d'esprit.

1. Le point de départ, tel que les équipes le décrivent

1.1 Le cycle actuel



Ce cycle a une qualité qu'il faut reconnaître avant de le critiquer : **il existe et il est décrit**. Une démonstration de mi-itération, un chiffrage avec le Product Owner, un contrôle de sécurité avant déploiement — beaucoup d'équipes au deuxième sprint n'ont pas cela.

Ce qu'il n'a pas, c'est un **point de jonction outillé**. Tout ce qui traverse la frontière traverse une boîte mail.

1.2 Le chiffre qui cadre tout le reste

Le collecteur d'indicateurs DORA (`scripts/ci/collect_dora.py`), mesuré le **2026-09-23** sur les **50 pipelines** des trente derniers jours (valeurs rejouées à l'identique le 2026-10-02), puis recalculé le **2026-10-05** sur **67 pipelines**, après la release 1.0.1 :

Indicateur DORA	2026-09-23	2026-10-05	Lecture au 2026-10-05
Fréquence de déploiement	0,1667 / j	0,2667 / jour	8 déploiements réussis, sur trois journées
Délai de mise en production	1,38 h	4,76 h (médiane)	8 observations ; le déclenchement est manuel, et le maximum (40,88 h) contient un week-end
Temps de rétablissement	2,14 h	2,21 h (médiane)	4 observations ; les deux nouvelles mesurent la relance d'un cluster
Taux d'échec des changements	66,67 %	60 %	9 échecs sur 15 tentatives, dont 2 comptés comme annulés et 3 sur un cluster arrêté

Ce tableau a changé le 2026-09-22, et c'est le fait marquant de l'itération. Il affichait jusque-là **0,0** déploiement par jour, deux `null` et 100 % d'échec sur 7 tentatives. La vague 0 de ce plan est donc largement derrière nous : un runner auto-hébergé a levé la contrainte de quota, et trois défauts d'accès au cluster ont été corrigés dans la foulée (détail en A0.1 et dans `MONITORING.md §9.1`).

`null` et **0** ne disent toujours pas la même chose, et le collecteur refuse de les confondre : un délai rendu en **0** afficherait la performance parfaite là où il n'y aurait pas eu de mise en production. La règle ne se voit plus dans cette sortie ; elle redevient visible dès qu'on interroge une fenêtre sans déploiement.

Conséquence directe sur ce plan : la question n'est plus « la chaîne aboutit-elle ? » mais « à quel prix ? ». Avec **six tentatives sur dix qui échouent** — dont, le 2026-10-05, trois sur un cluster arrêté par un redémarrage de Docker Desktop, sans rapport avec le code —, optimiser la vitesse du cycle reste prématuré : on fiabilise d'abord, on accélère ensuite. La vague 1 devient donc l'entrée réelle du plan, et les quatre chiffres ci-dessus deviennent une ligne de base — ce qu'ils ne pouvaient pas être tant qu'ils valaient `null`.

2. Les irritants, et ce qu'ils coûtent

Les huit irritants ci-dessous sont soit cités par les équipes, soit établis en lisant le dépôt. La colonne « établi par » distingue les deux, parce qu'un irritant vécu et un irritant constaté ne se traitent pas de la même façon : le premier a déjà l'adhésion de l'équipe, le second doit d'abord être partagé.

#	Irritant	Établi par	Ce qu'il coûte aujourd'hui
I1	Deux déploiements sur trois échouent depuis la CI	Dépôt — DORA	Requalifié le 2026-09-22 : la chaîne aboutit, elle n'est pas fiable. Au 2026-10-05 : 8 réussites sur 15, taux d'échec à 60 %. Les trois derniers échecs viennent de la plateforme (minikube arrêté), pas du code.
I2	Les CVE se découvrent chez Ops , après remise	Les deux sondages	Le premier déploiement de démonstration a été retardé. Retravail non planifié en fin d'itération, au pire moment.
I3	Les scans existent dans la CI mais ne bloquent rien	Dépôt	<code>trivy-fs</code> et les scans d'image tournent en <code>--exit-code 0</code> . Une image vulnérable est publiée comme une image saine. I2 semble traité alors qu'il ne l'est pas.
I4	Le déploiement est manuel , en commandes Docker	Sondage Ops	Non reproductible, non traçable, indisponible quand la personne qui sait ne l'est pas. Cité comme irritant n°1 par l'équipe Ops.
I5	Rupture de parité : HyperSQL en dev, PostgreSQL en prod	Croisement des deux sondages	Le code n'a jamais vu sa base de production. Le schéma est généré par Hibernate sans migration. Le back est plafonné à 1 replica.
I6	La frontière Dev↔Ops est un email	Sondage Ops	Aucune traçabilité, aucun horodatage, aucune réponse rapide à « quelle version tourne ? ».
I7	Trois artefacts dont une image tout-en-un	Sondage Dev	Couple les cadences front et back, empêche la montée en charge, triple la surface à scanner.
I8	Aucune compétence Kubernetes déclarée dans les deux équipes	Absence dans les deux sondages	La cible repose sur une compétence que personne n'a. Risque non pas de lenteur, mais d'incident non diagnosticable.

Trois irritants sont traités et il serait malhonnête de les recompter. I7 est réglé : l'image tout-en-un a été scindée au commit `2931df3` puis supprimée au commit `f125ef0`. I3 est réglé depuis le 2026-09-19 : les scans bloquent (A1.1). I2 l'est donc aussi, avec une réserve que le 2026-10-02 a mise au jour : l'une des portes, Dependency-Check, n'analysait aucun jar et « passait » sur du vide (A1.3). La colonne « ce qu'il coûte aujourd'hui » garde la description d'origine de ces irritants : c'est ce qui permet de mesurer l'écart.



1. La sécurité **arrête** le pipeline au lieu de le commenter.
2. Le déploiement en staging est **déclenché par le pipeline**, pas par une personne.
3. **Le seul acte humain restant est la décision de mettre en production**. Ce n'est pas une automatisation incomplète, c'est le point d'arrêt voulu : l'automatisation doit supprimer le geste répétitif, pas la décision.

4. Le plan, par vagues

Chaque action porte un identifiant, l'irritant traité, un porteur, un effort ($S \leq 1j$, $M \leq 3j$, $L > 3j$) et **une preuve d'atteinte** — ce qu'on regarde pour dire que c'est fait.

État des vagues au 2026-10-05. Le travail du 2026-10-02 a été fusionné le 2026-10-03 (PR #31 à #35) puis livré : pipelines #2909284076 (develop), #2912362926 (main) et #2913490784 (tag v1.0.1) verts sur leurs jobs automatiques, release 1.0.1 en production. Plus aucune action n'est « écrite sans avoir tourné ».

Vague	Actions faites	En partie	Pas commencées
0	A0.1, A0.2, A0.3 — close	—	—
1	A1.1, A1.2, A1.3	—	A1.4, A1.6 ; A1.5 n'est pas mesurable
2	A2.1	—	A2.2, A2.3, A2.4, A2.5
3	A3.3	—	A3.1, A3.2, A3.4, A3.5
4	A4.3	A4.2 (Release sans changelog)	A4.1, A4.4

Hors plan, deux chantiers ont avancé et changent la lecture des vagues 3 et 4 : **huit règles d'alerte Kibana** couvrent disponibilité, performance et sécurité, sans notification hors de Kibana (MONITORING.md §11) ; et les **traces de l'API** sont en service en staging et en production depuis le 2026-10-05 (§10 du même document).

Un enseignement nouveau de la release 1.0.1, d'où A1.6. Du 2 au 5 octobre, Docker Desktop a décroché ou redémarré quatre fois, laissant minikube à moitié arrêté : un runner figé (concurrent = 1) qui a retenu develop près d'une heure, deux terraform-plan en échec, trois deploy-production en échec. Aucun contrôle de la chaîne n'a failli ; c'est la plateforme. Le compte rendu est dans RELEASE.md §7.5.

Vague 0 — Faire aboutir la chaîne une fois — close le 2026-09-22

C'était le préalable à tout le reste : tant qu'elle n'était pas close, les vagues suivantes optimisaient quelque chose dont on ignorait si cela fonctionnait. Les trois actions sont réalisées ; elles restent inscrites ici avec leur preuve, parce qu'un plan dont on efface les lignes faites ne se relit plus.

ID	Action	Irritant	Porteur	Effort	Preuve d'atteinte
A0.1	Rétablir une capacité d'exécution CI : runner auto-hébergé sur le poste Ops, ou remise à niveau du quota GitLab	I1	Nico	M	Fait le 2026-09-22 — runner auto-hébergé enregistré sur le poste. Plus un pipeline ne s'arrête sur <code>ci_quota_exceeded</code>
A0.2	Obtenir un déploiement staging réussi depuis la CI, de bout en bout	I1	Nico + Temim	M	Fait le 2026-09-22 à 18:47 — <code>deploy-staging</code> en vert sur <code>develop</code> , après trois échecs le même jour
A0.3	Rejouer le collecteur DORA après A0.2	I1	Josefina	S	Fait le 2026-09-23 — <code>deployment_frequency</code> à 0,1667 et plus aucun indicateur à <code>null</code> (§1.2)

Ce que la vague 0 a coûté, et qu'il faut retenir. Le runner n'a pas suffi. Il a surtout rendu visibles trois défauts que personne n'avait pu rencontrer, faute de job allant assez loin : un kubeconfig de poste désignant `127.0.0.1`, inatteignable depuis un conteneur ; un RBAC d'agent qui ne couvrait aucun objet applicatif ; une variable `$STAGING_NAMESPACE` absente, qui faisait déployer dans `default` sans rien signaler. Les trois sont corrigés, le dernier par un garde-fou qui fait échouer le job. C'est la démonstration la plus nette du principe directeur de ce document : **une chaîne qu'on n'exécute pas ne révèle pas ses défauts, elle les conserve.**

Pourquoi A0.2 restait confiée à un binôme Ops + junior. C'était le premier déploiement réussi du projet : le faire à deux met Temim au contact du geste que personne dans l'équipe n'avait encore accompli, et donne à Nico un témoin de ce qui casse. Les trois échecs du 22 septembre, diagnostiqués l'un après l'autre avant la réussite de 18:47, sont précisément le matériel que ce binôme devait voir passer. La montée en compétence I8 commence ici, pas en vague 3.

⚠ **Close ne veut pas dire acquise.** Huit déploiements réussis sur quinze tentatives, sur trois journées : la chaîne aboutit, elle n'est pas fiable. Le bloquant suivant n'est plus l'exécution, c'est le taux d'échec — ce que traitent la vague 1 et A3.1.

Vague 1 — Rendre les contrôles contraignants

ID	Action	Irritant	Porteur	Effort	Preuve d'atteinte
A1.1	Passer les scans Trivy en mode bloquant sur HIGH, CRITICAL (trivy-fs, package-back, package-front)	I3, I2	Maïa	S	Fait le 2026-09-19. Une image porteuse d'une CVE critique fait échouer package-*. Depuis le 2026-10-03 (en CI) : le scan d'image a lieu avant le push, et chaque scan publie un rapport JSON en artefact (scripts/ci/trivy_scan.sh)
A1.2	Tenir un fichier d'exceptions daté et justifié : une entrée = un identifiant, un chemin, une raison, une date	I3	Maïa + Roubina	S	Fait le 2026-09-19 — . trivyignore.yaml, 7 entrées au 2026-10-02, bornées par chemin, revue au 2026-12-31 ; et depuis le 2026-10-02, suppressions.xml pour Dependency-Check : 12 CVE de Spring Framework, même échéance
A1.3	Rendre dependency-check-back bloquant au-delà d'un score CVSS convenu	I2	Sylvain	S	Bloquant depuis le 2026-09-19, effectif depuis le 2026-10-02 seulement. Le job n'analysait aucun jar (skipTestGroups) ; il en lit 82 — en CI depuis le 2026-10-03, rapport relu en artefact —, et a échoué en local sur 13 CVE réelles avant traitement. La preuve prévue — une dépendance vulnérable introduite exprès — n'a pas été jouée : c'est une vraie qui a fait la démonstration
A1.4	Miroiter les images de base dans le registry interne et n'y référencer qu'elles	I6, souhaite Ops C8	Nico	M	Aucun FROM ne pointe vers DockerHub ; le build passe DockerHub coupé
A1.5	Remplacer l'email par le pipeline comme canal : un déploiement se désigne par un tag d'image, jamais par un numéro dicté	I6	Roubina + Nico	S	Aucun échange de version par email sur une itération complète

ID	Action	Irritant	Porteur	Effort	Preuve d'atteinte
A1.6	Vérifier que le cluster répond (<code>kubectl get --raw /readyz</code>) en tête des jobs de déploiement ; passer le runner à <code>concurrent > 1</code>	I1	Nico	S	Un cluster arrêté fait échouer le job avec un message qui le nomme, avant tout <code>apply</code> ; un runner figé ne bloque plus tous les pipelines

A1.3 enseigne ce qu'une preuve d'atteinte doit regarder. Le job était bloquant et vert ; la case aurait pu être cochée le 2026-09-19. Il ne lisait rien. Une porte bloquante qui ne lit rien ne se distingue pas, à l'œil, d'une porte qui n'a rien trouvé — la preuve d'atteinte d'un contrôle doit donc inclure ce qu'il a regardé, pas seulement son verdict.

A1.1 tient en un caractère, et c'est précisément le piège. Le travail n'est pas la modification, c'est A1.2 : sans une liste d'exceptions tenue honnêtement, un scan bloquant devient un scan qu'on contourne, et l'équipe apprend à ignorer un signal rouge. Les deux actions sont indissociables ; les livrer séparément serait pire que ne rien faire.

Vague 2 — Réparer la parité d'environnement

ID	Action	Irritant	Porteur	Effort	Preuve d'atteinte
A2.1	Faire tourner les tests d'intégration du back contre un PostgreSQL réel en service de CI	I5	Sylvain	M	Fait le 2026-09-04 (commit <code>ffe1d39</code>) — <code>test-back</code> et <code>mutation-back</code> tournent sur un service <code>postgres:16-alpine</code> (QUALITY.md §7)
A2.2	Introduire Flyway ou Liquibase et retirer <code>ddl-auto</code> en environnement persistant	I5	Sylvain + Roubina	L	Une évolution d'entité produit un script de migration, pas une recreation de schéma
A2.3	Déployer PostgreSQL en <code>StatefulSet</code> + PVC, identifiants en <code>Secret</code>	I5	Nico	M	La base survit à la suppression du pod applicatif
A2.4	Lever le plafond de 1 replica sur le back et le vérifier sous charge k6	I5	Maïa + Temim	S	2 replicas servent des données cohérentes sous <code>k6-load</code>

ID	Action	Irritant	Porteur	Effort	Preuve d'atteinte
A2.5	Écrire et jouer la procédure de sauvegarde/restauration	I5	Nico	M	Une restauration effective, pas un <code>CronJob</code> jamais rejoué

L'ordre est contraint et ne se négocie pas. A2.2 avant A2.3 : mettre en place une base persistante en gardant `ddl-auto` revient à installer un mécanisme qui détruira des données à la première évolution d'entité. La commodité tolérable sur une base jetable devient un danger dès qu'elle persiste.

Vague 3 — Automatiser le déploiement et combler l'angle mort

ID	Action	Irritant	Porteur	Effort	Preuve d'atteinte
A3.1	Passer <code>deploy-staging</code> de <code>when: manual</code> à <code>when: on_success</code>	I4	Nico	S	Un merge sur <code>develop</code> met à jour staging sans intervention
A3.2	Tests post-déploiement en TestInfra — l'outil que l'équipe Ops maîtrise déjà	I4, I8	Maïa	M	Un déploiement fonctionnel mais cassé fait échouer le job
A3.3	Exécuter le collecteur DORA dans la CI au lieu de le lancer à la main	I1	Josefina	S	Fait le 2026-10-03 : job <code>dora-metrics</code> sur <code>develop</code> , <code>main</code> et les tags, artefact <code>reports/dora.json</code> (premier passage dans <code>#2909284076</code>). Il n'alimente pas Elasticsearch, injoignable depuis un job : le tableau de bord reste rempli à la main
A3.4	Montée en compétence Kubernetes : deux ateliers, un incident simulé, une astreinte en binôme	I8	Nico (anime), toute l'équipe	L	Chaque membre a diagnostiqué seul un pod en <code>CrashLoopBackOff</code> et déclenché un rollback

ID	Action	Irritant	Porteur	Effort	Preuve d'atteinte
A3.5	Documenter les décisions structurantes en ADR courtes	I8	Josefina	S	Une ADR par décision d'architecture non triviale

Sur A3.2 et le choix de TestInfra. Le réflexe serait d'écrire ces tests dans l'outillage du pipeline. On les écrit en TestInfra parce que l'équipe Ops le note « bon » et qu'elle sera celle qui les lira à 3 h du matin. Un test de production que son lecteur ne sait pas déchiffrer ne sera pas maintenu — et un test non maintenu finit désactivé.

Sur A3.4 et son critère de sortie. « Se former à Kubernetes » n'est pas un critère ; « avoir diagnostiqué seul un CrashLoopBackOff et déclenché un rollback » en est un. C'est la seule ligne du plan dont l'effort est en semaines, et c'est aussi celle qui protège de l'incident le plus coûteux : celui que personne ne sait lire.

Vague 4 — Réduire le risque de la mise en production

ID	Action	Irritant	Porteur	Effort	Preuve d'atteinte
A4.1	Déploiement progressif — canary ou blue/green	I4	Nico + Maïa	L	Une version fautive n'atteint qu'une fraction du trafic
A4.2	Changelog généré depuis les Conventional Commits	I6	Josefina	S	En partie : le job <code>release</code> a créé la Release GitLab <code>v1.0.1</code> le 2026-10-05, avec le commit, le pipeline et les images — pas encore la liste des changements
A4.3	Notification d'échec de déploiement sur un canal partagé	I6	Temim	S	Fait le 2026-09-29 (commit <code>44a1418</code>) — <code>notify-echec</code> et l' <code>after_script</code> des déploiements, vers <code>NOTIFY_WEBHOOK_URL</code> . Sans la variable, le message reste dans le journal du job
A4.4	Signature des images et attestation de provenance	I2	Maïa	M	Une image non signée est refusée au déploiement

5. Comment on saura que le plan a produit un effet

Quatre indicateurs DORA, plus quatre propres aux irritants d'Orion. La colonne « aujourd'hui » n'est pas une estimation : c'est ce qui est mesuré, ou l'indication franche qu'il n'y a pas de mesure.

Indicateur	Aujourd'hui	Après vague 1	Après vague 3	Comment on le mesure
Fréquence de déploiement	0,2667 / j (0,1667 au 2026-09-23)	> 0,33 / j	≥ 1 / j sur staging	collect_dora.py
Délai de mise en production	4,76 h (1,38 h)	< 1,38 h	< 1 j, sans clic	collect_dora.py
Temps de rétablissement	2,21 h (2,14 h), 4 cas	mesurable sur plus de 4 cas	< 1 h	collect_dora.py
Taux d'échec des changements	60 % (66,67 %)	< 50 %	< 15 %	collect_dora.py
Délai entre remise et retour CVE	minutes (le job échoue) depuis le 2026-09-19	minutes	idem	Durée du job package-*
Part d'opérations de déploiement manuelles	100 %	100 %	staging à 0 %	Comptage des gestes when: manual
Artefacts publiés par release	2 (3 avant f125ef0)	2	2	docker images du registry
Environnements où le code voit sa base de prod	1 (CI, depuis A2.1)	1	1	Présence du service PostgreSQL dans test-back

Trois honnêtetés à maintenir dans ce tableau. **Un indicateur sans donnée se rend null, jamais 0** — c'est la règle du collecteur, et l'afficher en 0 transformerait une absence de mesure en performance parfaite. **La part d'opérations manuelles reste à 100 % après la vague 1** : rendre un scan bloquant ne déploie rien, et le déclenchement des deux jobs de déploiement reste un clic. Une case qui ne bouge pas au bon moment est le signe que le tableau mesure quelque chose de réel.

Et la troisième, qui porte sur la colonne « aujourd'hui » elle-même : **ces valeurs reposent sur quinze tentatives étalées sur trois jours**. Elles constituent une ligne de base parce qu'il en faut une, pas parce qu'elles seraient stables. La cible « après vague 1 » d'un délai inférieur à 1,38 h — la ligne de base du 2026-09-23 — ne vaudra d'ailleurs que si elle se compare à un nombre d'observations comparable : la hausse à 4,76 h ne dit pas que la chaîne a ralenti, elle dit qu'un commit fusionné un samedi a été déployé le lundi. De même, trois des neuf échecs comptés au 2026-10-05 viennent d'un cluster arrêté : le taux d'échec mesure aussi la fiabilité du poste qui porte la chaîne, et A1.6 vise

précisément cette part. S'y ajoute une limite du collecteur, assumée et documentée en `MONITORING.md` §9.3 : un rollback qui a échoué compte quand même comme une annulation, donc le taux d'échec est plutôt au-dessus de la réalité qu'en dessous.

6. Ce que le plan ne traite pas, et pourquoi

- **L'authentification de l'application.** MicroCRM n'en a aucune. C'est un sujet produit, pas un sujet de cycle de release ; il doit être ouvert ailleurs, et avant tout usage réel.
 - **Le passage au cloud.** L'architecture reste locale. Le chiffage AWS/Azure n'existe pas, et un plan qui promettrait une bascule sans coût connu ne serait pas un plan.
 - **La sécurisation de la stack ELK.** `xpack.security` est désactivé et Kibana n'a pas d'Ingress ; assumé pour une stack de démonstration locale, inacceptable ailleurs. À rouvrir si elle sort du poste.
 - **La vérification des sources Mermaid dupliquées.** Rien ne compare les blocs des documents et les fichiers `docs/schemas/*.mmd`. Le coût d'un test dépasse aujourd'hui celui d'une relecture — mais c'est un choix, pas un oubli.
-

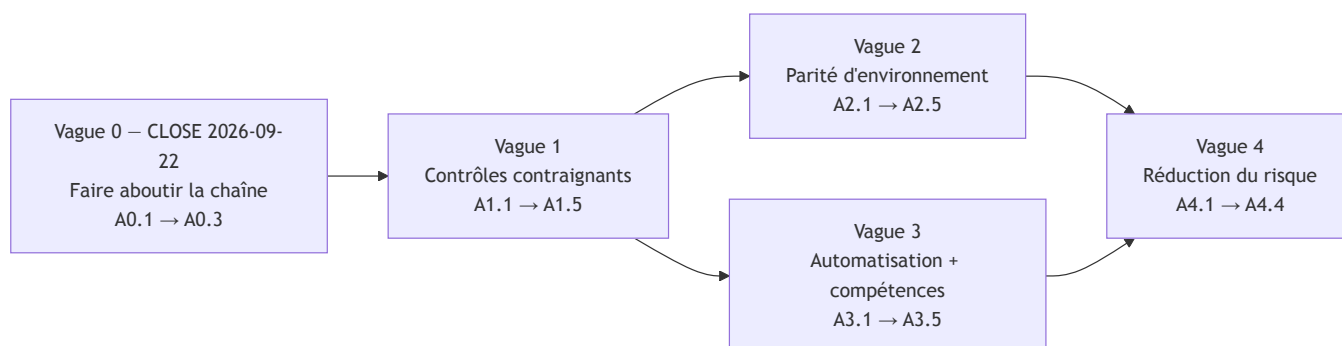
7. Les risques du plan lui-même

Un plan qui n'énonce pas ses propres risques demande qu'on lui fasse confiance.

Risque	Probabilité	Impact	Ce qu'on fait
A1.1 sans A1.2 : le blocage devient un obstacle qu'on contourne	élevée	fort	Les deux actions sont livrées ensemble, et la liste d'exceptions est relue à chaque itération
La charge repose sur deux personnes côté Ops	élevée	fort	A3.2 et A3.4 transfèrent volontairement du travail vers l'équipe Dev ; A0.2 est en binôme dès le premier jour
La vague 2 déborde. Flyway sur une base existante est un chantier, pas une tâche	moyenne	moyen	A2.1 seule apporte déjà l'essentiel du bénéfice et se livre indépendamment
La compétence Kubernetes ne monte pas faute de temps dédié	moyenne	critique	A3.4 porte un critère de sortie observable ; sans lui, le sujet se reporte indéfiniment

Risque	Probabilité	Impact	Ce qu'on fait
Le runner auto-hébergé tombe ou change de poste — la contrainte a été déplacée, pas supprimée	moyenne	fort	A0.1 l'a mis sous le contrôle de l'équipe ; reste à ne pas dépendre d'une seule machine, et de qui l'allume
Josefina termine son stage avant la fin des actions qui lui sont confiées	certaine à terme	faible	Ses actions sont toutes en effort S et documentées par des ADR (A3.5)

8. Ordre d'exécution résumé



Les vagues 2 et 3 sont **parallélisables** : la première est un chantier back porté par Sylvain, la seconde un chantier plateforme porté par Nico. Elles se rejoignent avant la vague 4, parce qu'un déploiement progressif sur une base non persistante ne prouverait rien.

La vague 0, elle, ne se parallélisait pas — c'était le seul point du plan sur lequel il n'y avait pas d'arbitrage possible, et elle est close depuis le 2026-09-22. Ce qu'elle a démontré tient en une phrase, qui vaut pour la suite : **une chaîne qu'on n'exécute pas ne révèle pas ses défauts, elle les conserve**. Trois défauts attendaient derrière le quota ; rien ne dit que la vague 1 n'en découvrira pas d'autres en rendant les contrôles bloquants.